A non-expert's gentle intro about formal language and Ai4Math

L03x: Logic extension

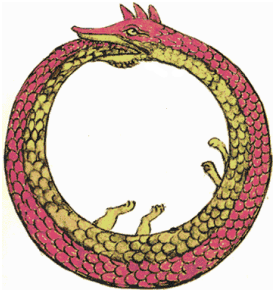
COMP2711 Discrete Mathematical Tools

Mingxun Zhou (mingxunz@ust.hk)

SECTION 1

Why formal logic?

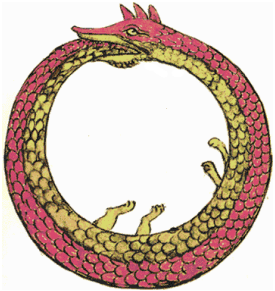
The liar paradox



"This sentence is false."

Source: Ouroboros image: Wikimedia Commons, public domain.

The liar paradox



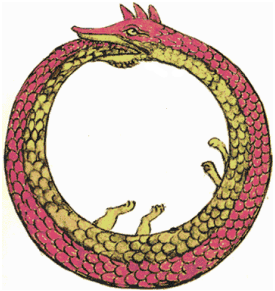
"This sentence is false."

If true: what it says is correct, so it is false.

If false: what it says is wrong, so it is true.

Source: Ouroboros image: Wikimedia Commons, public domain.

The liar paradox



"This sentence is false."

If true: what it says is correct, so it is false.

If false: what it says is wrong, so it is true.

The sentence turns back and **talks about itself**.

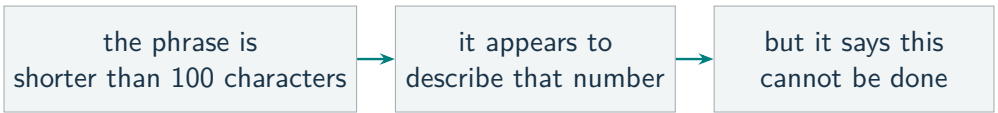
Source: Ouroboros image: Wikimedia Commons, public domain.

What does “described” mean?

“the smallest natural number that cannot be described in fewer than 100 characters”

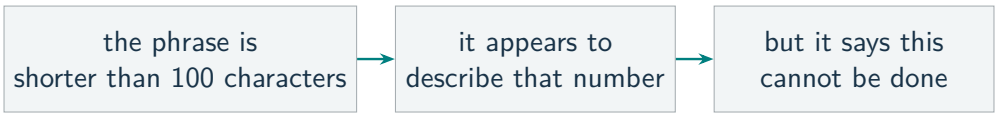
What does “described” mean?

“the smallest natural number that cannot be described in fewer than 100 characters”



What does “described” mean?

“the smallest natural number that cannot be described in fewer than 100 characters”



English never fixed what counts as a **description**.

The Big Number Duel

Source: Numberphile; image links to the video.

The Daddy of Big Numbers

Numberphile, 15:26

Watch for two moments:

When does the contest stop naming larger numbers directly?

What new language does Rayo introduce?

Busy Beaver: A Python's Perspective

Fix one precise, deterministic, general-purpose Python-like model.

Source: Python-style runtime analogue of Radó's Busy Beaver function.

Busy Beaver: A Python's Perspective

Fix one precise, deterministic, general-purpose Python-like model.

For a length limit n, let F_n be the program that

runs the **longest**, but **eventually halts**,
among all programs with Length$(F_n) \leq n$.

Source: Python-style runtime analogue of Radó's Busy Beaver function.

Busy Beaver: A Python's Perspective

Fix one precise, deterministic, general-purpose Python-like model.

For a length limit n, let F_n be the program that

runs the **longest**, but **eventually halts**,
among all programs with $\text{Length}(F_n) \leq n$.

$$\text{PyBB}(n) = \text{RunningTime}(F_n)$$

For $n = 1000$: the slowest program of at most 1000 bytes
among those that do not run forever.

Source: Python-style runtime analogue of Radó's Busy Beaver function.

Busy Beaver: A Python's Perspective

For any particular program F', exactly one statement is true:

Source: The empty program halts, so the set being maximized is nonempty.

Busy Beaver: A Python's Perspective

For any particular program F', exactly one statement is true:

$\text{Halt}(F') = \text{True}$	$\text{Halt}(F') = \text{False}$
F' eventually stops	F' runs forever

No ambiguity: the truth value is well-defined, even if we cannot always find it.

Source: The empty program halts, so the set being maximized is nonempty.

Busy Beaver: A Python's Perspective

For any particular program F', exactly one statement is true:

$\text{Halt}(F') = \text{True}$	$\text{Halt}(F') = \text{False}$
F' eventually stops	F' runs forever

No ambiguity: the truth value is well-defined, even if we cannot always find it.

$$\text{PyBB}(1000) = \max\left\{ \text{RunningTime}(F) \;\middle|\; \begin{matrix} \text{Length}(F) \leq 1000 \\ \text{Halt}(F) = \text{True} \end{matrix} \right\}.$$

A well-defined number, once the computing model is precise.

This is why Turing machines matter: they give us a precise model of computation.

Source: The empty program halts, so the set being maximized is nonempty.

Busy Beaver: A Python's Perspective

What would "computable" mean?

Source: Alan Turing, "On Computable Numbers," 1936.

Busy Beaver: A Python's Perspective

What would "computable" mean?

There exists one finite-length program C that always halts and does this:

$$\boxed{n \longrightarrow C(n) = \text{PyBB}(n)}$$

Source: Alan Turing, "On Computable Numbers," 1936.

Busy Beaver: A Python's Perspective

What would "computable" mean?

There exists one finite-length program C that always halts and does this:

$$\boxed{n \longrightarrow C(n) = \text{PyBB}(n)}$$

If C existed, then for any program P we could compute $T = C(\text{Length}(P))$ and run P for T steps:

$$\text{stops by step } T \Rightarrow \text{halts} \qquad \text{still running} \Rightarrow \text{never halts.}$$

Source: Alan Turing, "On Computable Numbers," 1936.

Busy Beaver: A Python's Perspective

What would "computable" mean?

There exists one finite-length program C that always halts and does this:

$$\boxed{n \longrightarrow C(n) = \text{PyBB}(n)}$$

If C existed, then for any program P we could compute $T = C(\text{Length}(P))$ and run P for T steps:

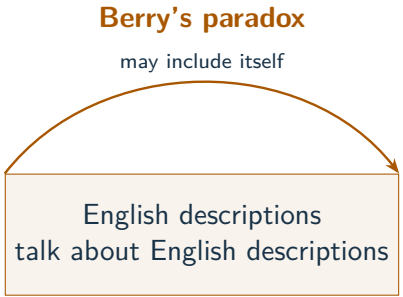
stops by step $T \Rightarrow$ halts still running $\Rightarrow$ never halts.

The halting problem is undecidable (Turing, 1936).
PyBB(n) **is well-defined, but the function is not computable.**

Interested? **Theory of Computation** studies these limits.

Source: Alan Turing, "On Computable Numbers," 1936.

Rayo's idea: separate the language levels

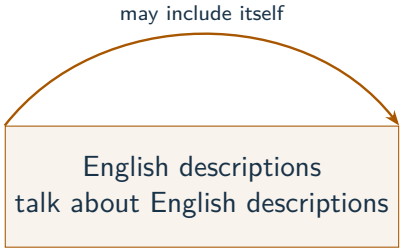


"described" has no fixed boundary.

Source: Agustín Rayo, "Big Number Duel."

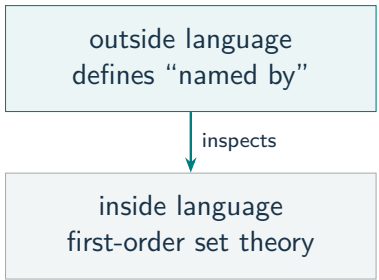
Rayo's idea: separate the language levels

Berry's paradox



"described" has no fixed boundary.

Rayo's construction



The outside description is **not an inside entry**.

Source: Agustín Rayo, "Big Number Duel."

The inside language is controlled but expressive

Controlled

- fixed finite alphabet
- exact grammar and interpretation
- fewer than 10^{100} symbols

Expressive enough to name numbers

In set theory,

$$0 = \varnothing, \qquad 1 = \{\varnothing\}.$$

Source: Agustín Rayo, "Big Number Duel."

The inside language is controlled but expressive

Controlled

- fixed finite alphabet
- exact grammar and interpretation
- fewer than 10^{100} symbols

only **finitely many** formulas qualify

Expressive enough to name numbers

In set theory,

$$0 = \varnothing, \qquad 1 = \{\varnothing\}.$$

A first-order formula for 1 is

$$\forall y \big(y \in x \leftrightarrow \forall z \, \neg (z \in y) \big).$$

y is in x exactly when y is empty.

Source: Agustín Rayo, "Big Number Duel."

The inside language is controlled but expressive

Controlled

- fixed finite alphabet
- exact grammar and interpretation
- fewer than 10^{100} symbols

only **finitely many** formulas qualify

Expressive enough to name numbers

In set theory,

$$0 = \varnothing, \qquad 1 = \{\varnothing\}.$$

A first-order formula for 1 is

$$\forall y \big(y \in x \leftrightarrow \forall z \, \neg (z \in y) \big).$$

y is in x exactly when y is empty.

It can encode arithmetic and programs—but not use the outer predicate **"named by a short first-order formula"** as an inside symbol.

Source: Agustín Rayo, "Big Number Duel."

Which formulas name one number?

A legal formula is not automatically a name.

$\varphi(x)$ fits no natural number	$\varphi(x)$ fits **exactly one** number	$\varphi(x)$ fits many numbers

Which formulas name one number?

A legal formula is not automatically a name.

$\varphi(x)$ fits no natural number	$\varphi(x)$ fits **exactly one** number	$\varphi(x)$ fits many numbers

Only the middle case counts as an **eligible name**.

Which formulas name one number?

A legal formula is not automatically a name.

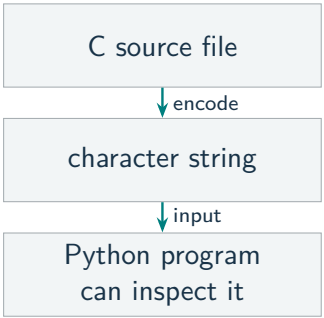
$\varphi(x)$ fits no natural number	$\varphi(x)$ fits **exactly one** number	$\varphi(x)$ fits many numbers

Only the middle case counts as an **eligible name**.

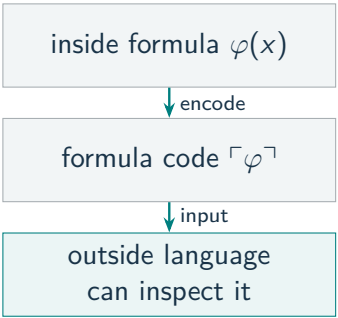
Syntax tells us which formulas are legal.
Interpretation tells us what they name.

Step 2: talk about the language from outside

Programming analogy



Logic analogy

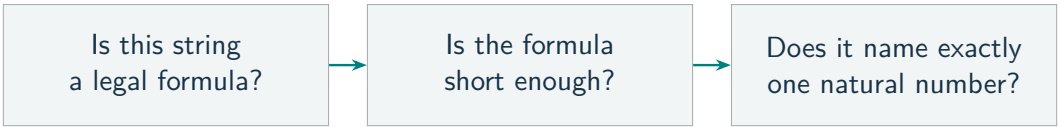


Step 2: talk about the language from outside

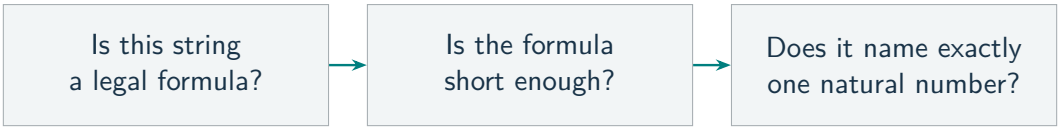
Written code becomes **data** for another language.

The winning description remains **outside** the collection it ranks.

What can the outside language say?



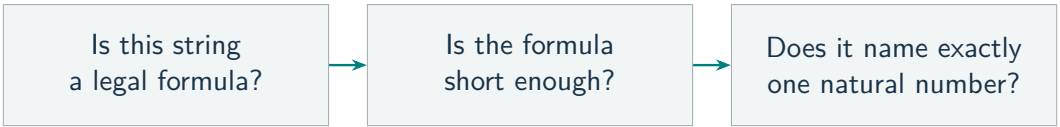
What can the outside language say?



To test the third question, the outside language defines

$\mathrm{Sat}(\ulcorner\varphi\urcorner, s)$: "formula φ is true under assignment s."

What can the outside language say?



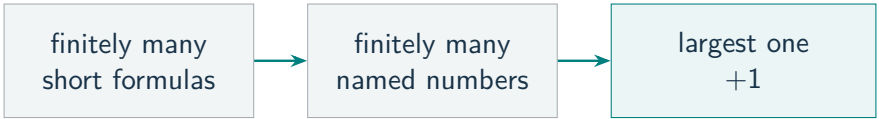
To test the third question, the outside language defines

$\mathrm{Sat}(\ulcorner\varphi\urcorner, s)$: "formula φ is true under assignment s."

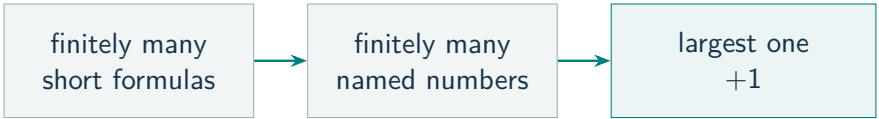
Now "described" has a **precise meaning**.

Defining this truth condition requires a stronger outside language.
Rayo uses **second-order set theory**.

Rayo's winning move

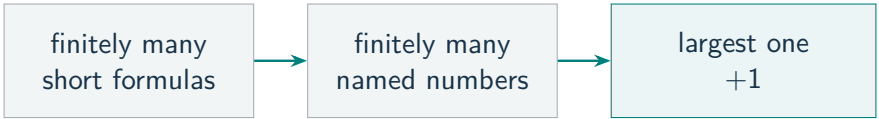


Rayo's winning move



Rayo's number $= 1 + \max\{m : m \text{ has an eligible name}\}$.

Rayo's winning move



Rayo's number $= 1 + \max\{m : m \text{ has an eligible name}\}$.

Under the fixed standard interpretation:
a **precise** contest and a well-defined winner.

Well-defined does not mean easy to compute.

SECTION 2

Logic systems and limits

First-order logic

Variables range over **objects**.

In a graph:
x, y, z are vertices.
$E(x, y)$ means adjacent.

A remarkable proof system

Formal proof $\Longrightarrow$ true in every interpretation

sound

First-order logic

Variables range over **objects**.

In a graph:
x, y, z are vertices.
$E(x, y)$ means adjacent.

A remarkable proof system

Formal proof $\Longrightarrow$ true in every interpretation

sound

True in every interpretation $\Longrightarrow$ a formal proof exists

complete

Gödel's 1929 result: first-order logic is complete.

A limit of the graph language

longer graph $\Rightarrow$ longer paths

First-order logic can ask for a path of length

$1, 2, 3, \ldots, k$

Intuition: one fixed formula cannot keep increasing k.

Source: Claim is for finite undirected graphs in the adjacency-only vocabulary $\{E\}$.

A limit of the graph language

longer graph $\Rightarrow$ longer paths

First-order logic can ask for a path of length

$1, 2, 3, \ldots, k$

Intuition: one fixed formula cannot keep increasing k.

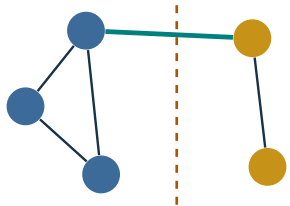
No single first-order sentence expresses **connectivity** for every finite undirected graph size.

Source: Claim is for finite undirected graphs in the adjacency-only vocabulary $\{E\}$.

Second-order connectivity

Every nonempty proper vertex set has an edge leaving it.

Second-order variables may range over sets (predicates) and relations.



every cut has a crossing edge

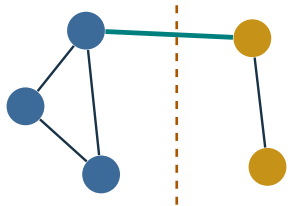
$$\forall X \Big[((\exists x\, X(x)) \wedge (\exists y\, \neg X(y))) \rightarrow \exists u \exists v\, (X(u) \wedge \neg X(v) \wedge E(u,v)) \Big].$$

$X(u), \neg X(v)$	opposite sides
$E(u,v)$	a crossing edge

Second-order connectivity

Every nonempty proper vertex set has an edge leaving it.

Second-order variables may range over sets (predicates) and relations.



every cut has a crossing edge

$$\forall X \Big[((\exists x\, X(x)) \wedge (\exists y\, \neg X(y))) \rightarrow \exists u \exists v\, (X(u) \wedge \neg X(v) \wedge E(u,v)) \Big].$$

$X(u), \neg X(v)$	opposite sides
$E(u,v)$	a crossing edge

One sentence works for every finite graph size.

Expressiveness has a price

First-order logic

Quantified variables range over **objects in the chosen domain**

Cannot quantify over arbitrary subsets, predicates, or relations

Effective, sound and complete proof calculus

Full second-order logic

Variables may also range over **subsets, predicates, and relations**

More expressive

Expressiveness has a price

First-order logic

Quantified variables range over **objects in the chosen domain**

Cannot quantify over arbitrary subsets, predicates, or relations

Effective, sound and complete proof calculus

Full second-order logic

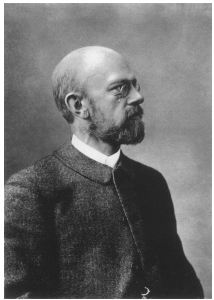
Variables may also range over **subsets, predicates, and relations**

More expressive

No effective, sound and complete proof calculus

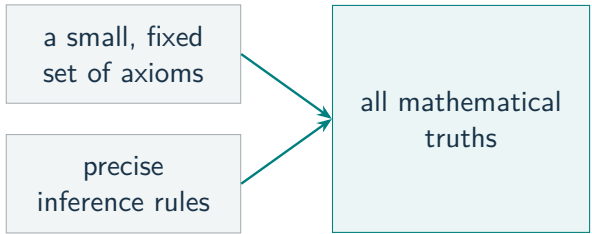
More expressive does not mean better for every task.

Hilbert's program



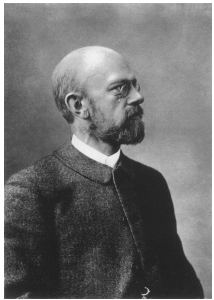
David Hilbert

Can one formal foundation secure all mathematics?



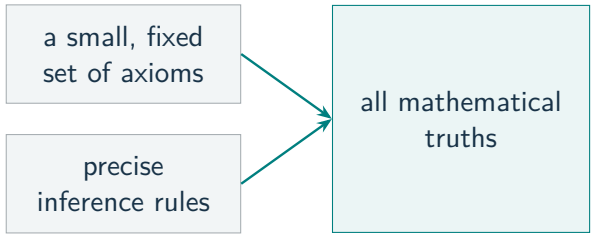
Source: Background: Wikipedia, "Hilbert's program" and "Foundational crisis of mathematics." Photo: Wikimedia Commons, public domain.

Hilbert's program



David Hilbert

Can one formal foundation secure all mathematics?



Complete Every true statement can be proved.

Consistent Never prove both P and $\neg P$.

Source: Background: Wikipedia, "Hilbert's program" and "Foundational crisis of mathematics." Photo: Wikimedia Commons, public domain.

Gödel's incompleteness theorem

Kurt Gödel

A system S that is consistent, effectively specified, and strong enough for integer arithmetic

true statements about integers

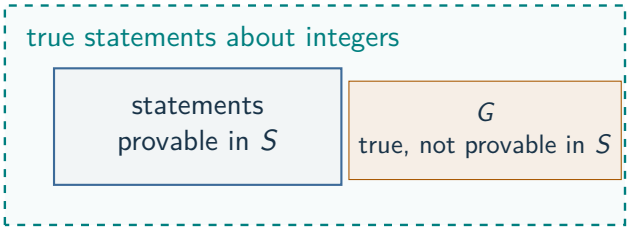
statements provable in S

Source: Gödel, 1931. Photo: Wikimedia Commons, public domain.

Gödel's incompleteness theorem

Kurt Gödel

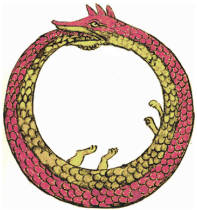
A system S that is consistent, effectively specified, and strong enough for integer arithmetic



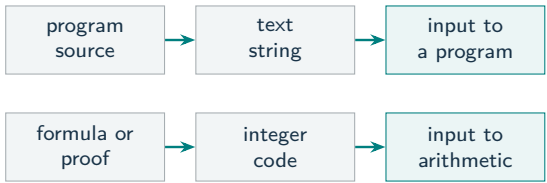
There is a true arithmetic statement beyond the system's proofs.

Source: Gödel, 1931. Photo: Wikimedia Commons, public domain.

The self-reference idea



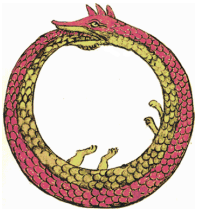
Gödel numbering



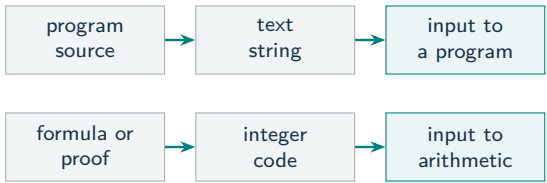
Arithmetic can now talk about codes of its own proofs.

Source: Ouroboros image: Wikimedia Commons, public domain.

The self-reference idea



Gödel numbering

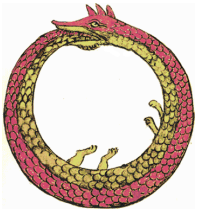


Arithmetic can now talk about codes of its own proofs.

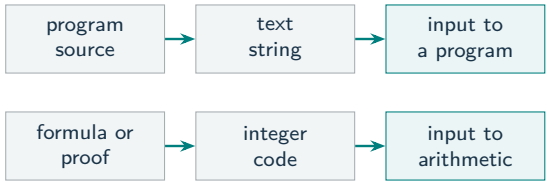
G says, roughly: **"G has no proof in S."**

Source: Ouroboros image: Wikimedia Commons, public domain.

The self-reference idea



Gödel numbering



Arithmetic can now talk about codes of its own proofs.

G says, roughly: **"G has no proof in S."**

If S is consistent, G is true but not provable in S.

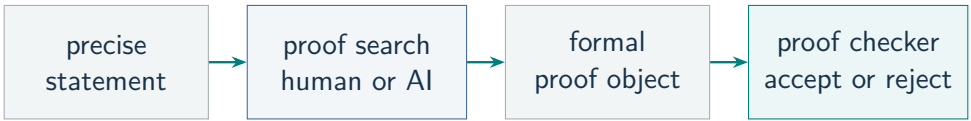
Source: Ouroboros image: Wikimedia Commons, public domain.

SECTION 3

Automated reasoning and AI

What is an automated proof?

A computer can perform two different jobs.

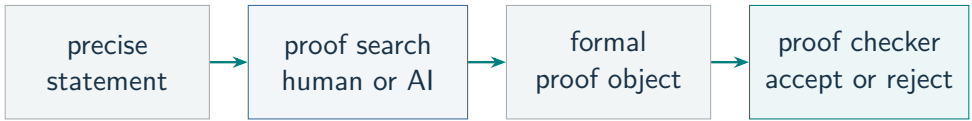


Search can be creative
Guess ideas, methods, and intermediate steps.

Checking must be exact
Verify every step against fixed rules.

What is an automated proof?

A computer can perform two different jobs.



Search can be creative
Guess ideas, methods, and intermediate steps.

Checking must be exact
Verify every step against fixed rules.

We do not have to trust who found the proof.
We can **re-check the proof object**.

What is Lean?

a proof assistant

Language State definitions and theorems precisely.

Assistant Construct a proof with human or automated help.

Kernel Check every proof step against fixed rules.

Source: Official Lean logo and system reference: lean-lang.org.

What is Lean?

a proof assistant

Language State definitions and theorems precisely.

Assistant Construct a proof with human or automated help.

Kernel Check every proof step against fixed rules.

mathematical idea $\longrightarrow$ Lean statement and proof $\longrightarrow$ **machine-verifiable theorem**

A leading standard for formal, machine-verifiable mathematics.

Source: Official Lean logo and system reference: lean-lang.org.

Modus ponens in Lean

```
theorem modus_ponens
    (p q : Prop)
    (hp : p)
    (h : p -> q) : q := by
  exact h hp
```

Source: Lean is based on dependent type theory, not simply first- or second-order logic.

Modus ponens in Lean

```
theorem modus_ponens
    (p q : Prop)
    (hp : p)
    (h : p -> q) : q := by
  exact h hp
```

`hp : p, h : p -> q` $\Longrightarrow$ `h hp : q`

The L03 rule, written as a program.

Source: Lean is based on dependent type theory, not simply first- or second-order logic.

Modus ponens in Lean

```
theorem modus_ponens
    (p q : Prop)
    (hp : p)
    (h : p -> q) : q := by
  exact h hp
```

`hp : p, h : p -> q` $\Longrightarrow$ `h hp : q`

The L03 rule, written as a program.

Kernel checks

type-checks the proof term
relative to declared axioms

Humans check

intended theorem, assumptions,
meaning

Source: Lean is based on dependent type theory, not simply first- or second-order logic.

Why we want fully machine-verifiable proofs?

Yitang Zhang

Famous for:
2013 breakthrough toward the twin-prime conjecture

Source: Wilkinson, 2015; WQED, *Counting from Infinity*; historical responsibility remains disputed.

Why we want fully machine-verifiable proofs?

Yitang Zhang

Famous for:
2013 breakthrough toward the twin-prime conjecture

After his PhD No academic position; he worked outside academia, including at Subway.

Source: Wilkinson, 2015; WQED, *Counting from Infinity*; historical responsibility remains disputed.

Why we want fully machine-verifiable proofs?

Yitang Zhang

Famous for:
2013 breakthrough toward the twin-prime conjecture

After his PhD No academic position; he worked outside academia, including at Subway.

Jacobian-conjecture thesis Under T. T. Moh, Zhang studied a weak form of the conjecture.

A faulty dependency One route relied on a theorem attributed to Moh that was later found faulty. They parted unhappily; accounts of responsibility differ.

Source: Wilkinson, 2015; WQED, *Counting from Infinity*; historical responsibility remains disputed.

Why we want fully machine-verifiable proofs?

Yitang Zhang

Famous for:
2013 breakthrough toward the twin-prime conjecture

After his PhD No academic position; he worked outside academia, including at Subway.

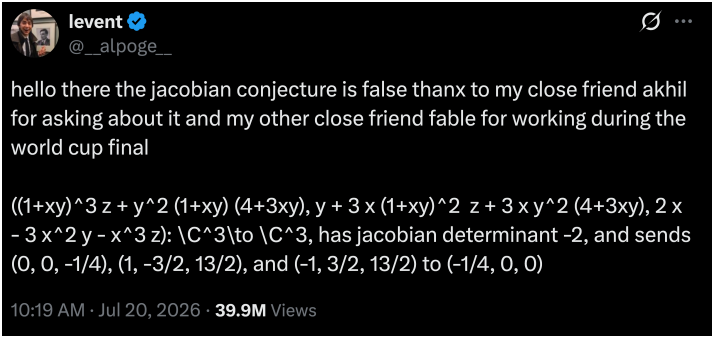
Jacobian-conjecture thesis Under T. T. Moh, Zhang studied a weak form of the conjecture.

A faulty dependency One route relied on a theorem attributed to Moh that was later found faulty. They parted unhappily; accounts of responsibility differ.

Lesson: Human-written mathematics contains errors—and always will.
One hidden dependency can invalidate years of work.

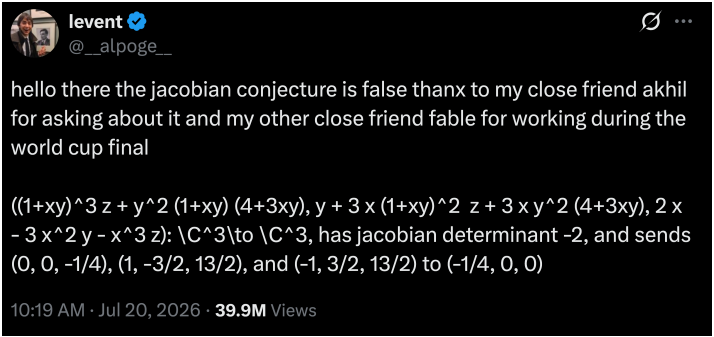
Source: Wilkinson, 2015; WQED, *Counting from Infinity*; historical responsibility remains disputed.

Why we want fully machine-verifiable proofs?



Source: Alpöge, X, 20 July 2026; Tao, 21 July 2026. Case $n = 2$ remains open.

Why we want fully machine-verifiable proofs?



July 2026: Alpöge + Claude Fable announced an explicit dimension-3 counterexample—an exact, directly checkable AI-assisted result.

Source: Alpöge, X, 20 July 2026; Tao, 21 July 2026. Case $n = 2$ remains open.

AI4Math in 2026

20 May	**Unit distances:** an OpenAI model finds a disproof; mathematicians check it.
21 July	**Jacobian conjecture:** Alpöge and Fable give an exact counterexample in dimension 3.
1 Aug.	**Ten reported advances:** Astra produces results across several fields, each with a Lean certificate.
4 Sept.	**Prime gaps:** OpenAI reports that Astra lowers the bound from 240 to 186.
8 Sept.	**Navier–Stokes:** OpenAI releases a proposed finite-time blowup proof and a Lean formalization.

Major announcements are now arriving **days apart**.

Source: OpenAI, 20 May, 1 Aug., 4 Sept., and 8 Sept. 2026; Tao, 21 July 2026.

8 September: Navier–Stokes

The Navier–Stokes equations describe how fluids move.

Can a smooth three-dimensional flow develop a **singularity** in finite time? This is a famous open question.

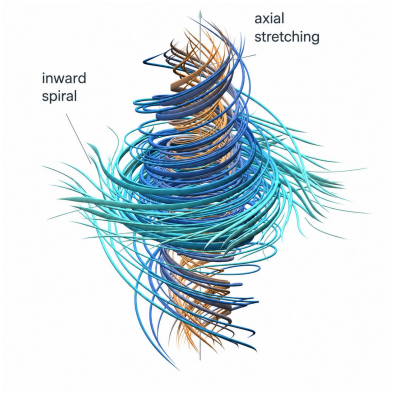


Illustration of the proposed blowup geometry.

Source: OpenAI, "On the Navier–Stokes Millennium Prize Problem," 8 Sept. 2026; OpenAI, *Finite Time Blowup for Navier–Stokes*; CMI prize rules.

8 September: Navier–Stokes

The Navier–Stokes equations describe how fluids move.

Can a smooth three-dimensional flow develop a **singularity** in finite time? This is a famous open question.

On 8 September 2026, OpenAI released an AI-generated proposed solution: such a singularity can occur.

The release includes a paper and a Lean-formalized proof.

Expert review and Clay recognition remain separate processes.

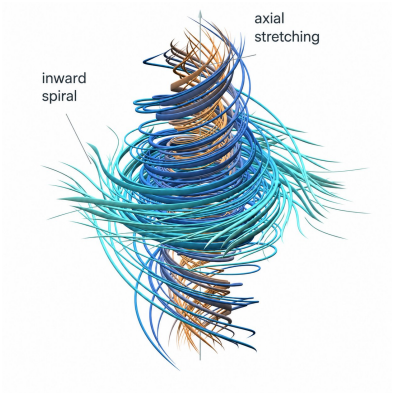


Illustration of the proposed blowup geometry.

Source: OpenAI, "On the Navier–Stokes Millennium Prize Problem," 8 Sept. 2026; OpenAI, *Finite Time Blowup for Navier–Stokes*; CMI prize rules.

Fast results need careful labels

Exact counterexample
Jacobian conjecture, $n \geq 3$
Check the explicit polynomial map.

New proposed resolution
Navier–Stokes blowup
Paper + Lean proof; expert review remains separate.

Formal certificate
Astra's ten reported advances
Lean checks the precise formal claims.

Contested claim
A complex structure on S^6
Not independently established as of 9 Sept. 2026.

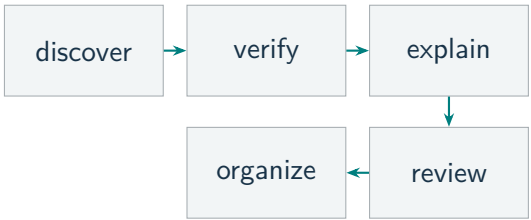
Discovery can accelerate. **Verification and interpretation still matter.**

Source: OpenAI, 1 Aug. and 8 Sept. 2026; Tao, 21 July 2026; circulated S^6 manuscript, Aug. 2026.

Proof abundance

AI may produce proofs faster than mathematicians can absorb them.

Terence Tao

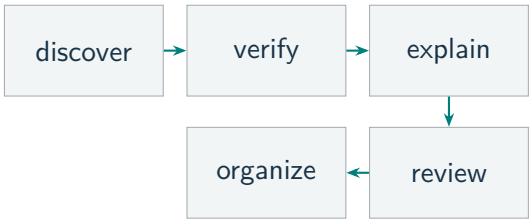


Source: Terence Tao, "Palomar – a registry of Lean verified mathematics," 18 Aug. 2026. Photo: Gert-Martin Greuel/MFO, CC BY-SA 2.0 DE.

Proof abundance

AI may produce proofs faster than mathematicians can absorb them.

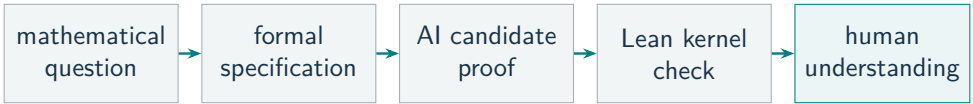
Terence Tao



Palomar records formal statements, dependencies, certificates, and review status.

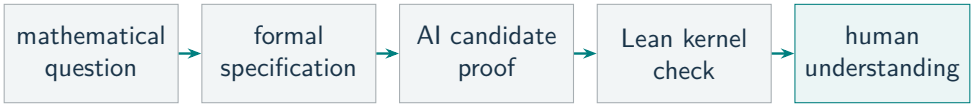
Source: Terence Tao, "Palomar – a registry of Lean verified mathematics," 18 Aug. 2026. Photo: Gert-Martin Greuel/MFO, CC BY-SA 2.0 DE.

Formal mathematics and AI



Source: Recommended viewing: Terence Tao, *Mathematics in the Age of AI*, ICM 2026 public lecture.

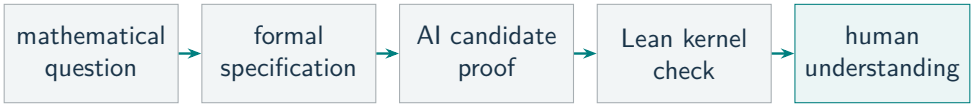
Formal mathematics and AI



AI scales **proof search**. Logic scales **proof checking**.

Source: Recommended viewing: Terence Tao, *Mathematics in the Age of AI*, ICM 2026 public lecture.

Formal mathematics and AI



AI scales **proof search**. Logic scales **proof checking**.

Natural language can hide **ambiguity and circularity**.
Formal language makes **syntax, meaning, and dependencies** explicit.

Why logic is fascinating,
and why it is so useful for AI development.

Source: Recommended viewing: Terence Tao, *Mathematics in the Age of AI*, ICM 2026 public lecture.